LAPORAN TUGAS
KLASIFIKASI NAIVE BAYES & SVM

Mata Kuliah Machine Learning – Pertemuan 8

Kelas : TI-C5 | Kelompok : 5

Anggota Kelompok:
1. Muhammad Irfan Nuha (434241079)
2. Bunga Lala Sal Sabila (434241066)

Tugas Kelompok

---

1. Pendahuluan

Tugas ini melanjutkan UTS kelompok kami. Dataset dan seluruh tahapan dibuat **sama dengan UTS**: dataset *Airline Passenger Satisfaction* (103.904 data penumpang) diproses melalui preprocessing, transformasi Min-Max, split 80% training dan 20% testing, serta seleksi fitur Chi-Square dan ANOVA F. Bedanya, kali ini klasifikasi dilakukan dengan **Naive Bayes** dan **SVM**, lalu hasilnya dibandingkan dengan **Decision Tree** dari UTS. Metode dengan akurasi tertinggi kemudian dibuatkan antarmuka (UI) berbasis **Streamlit** sehingga bisa dipakai untuk memprediksi kepuasan penumpang secara langsung.

2. Metode Klasifikasi

Naive Bayes (GaussianNB) menghitung peluang sebuah data masuk ke tiap kelas memakai Teorema Bayes, lalu memilih kelas dengan peluang terbesar. Disebut *naive* karena menganggap semua fitur saling independen, dan versi Gaussian mengasumsikan tiap fitur numerik berdistribusi normal. Metode ini sangat cepat, tetapi asumsinya jarang terpenuhi sempurna di data nyata.

SVM (Support Vector Machine) mencari *hyperplane* pemisah dua kelas dengan margin selebar mungkin. Dengan **kernel RBF**, data dipetakan ke ruang berdimensi lebih tinggi sehingga SVM mampu membentuk batas keputusan yang melengkung (non-linear). Kelemahannya, waktu pelatihan bertambah cepat seiring banyaknya data.

Decision Tree (dari UTS) membagi data secara bertahap berdasarkan aturan if-else pada fitur, membentuk struktur pohon. Model ini mudah ditafsirkan namun rawan *overfitting*.

3. Tahapan Program (Sama dengan UTS)

Program utama adalah **klasifikasi_434241079.py**. Berikut library dan konfigurasi yang dipakai:

py klasifikasi_434241079.py

```python
import os
import time
import warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import joblib
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder, MinMaxScaler
from sklearn.model_selection import train_test_split
from sklearn.feature_selection import SelectKBest, chi2, f_classif
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import (confusion_matrix, accuracy_score,
                             precision_score, recall_score, f1_score)

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
OUT_DIR  = os.path.join(BASE_DIR, "output")
os.makedirs(OUT_DIR, exist_ok=True)
TARGET   = "satisfaction"
K_FITUR  = 10
```

Kode 1. Import library dan konfigurasi

Bagian awal program identik dengan UTS. Data dibaca, kolom *id* dibuang, lalu dilakukan deteksi dan handling **missing value** (310 sel kosong di *Arrival Delay in Minutes* diisi median), pengecekan **duplikasi** (tidak ada), dan **capping outlier** dengan metode IQR. Kolom kategorikal di-encode menjadi angka, kemudian seluruh fitur dinormalisasi **Min-Max** ke rentang [0, 1] dan dibagi **80% training (83.123 data) : 20% testing (20.781 data)** secara stratified.

py klasifikasi_434241079.py

```
    # ---------------- 1. INPUT ----------------
    garis("1. INPUT DATA")
    df = pd.read_csv(os.path.join(BASE_DIR, "train.csv"))
    df.columns = df.columns.str.strip()
    df = df.drop(columns=[c for c in ["Unnamed: 0", "id"] if c in df.columns])
    print(f"Ukuran data : {df.shape[0]} baris x {df.shape[1]} kolom")
    print(df.iloc[:5, :6].to_string(index=False))
    print(f"... (+{df.shape[1] - 6} kolom lainnya)")

    # ---------------- 2. PREPROCESSING ----------------
    garis("2. PREPROCESSING")
    miss = df.isna().sum()
    print(f">> Missing value : {int(miss.sum())} sel "
          f"({', '.join(f'{k}={v}' for k, v in miss[miss > 0].items()) or '-'})")
    for c in df.select_dtypes(include=[np.number]).columns:
        df[c] = df[c].fillna(df[c].median())
    for c in df.select_dtypes(include="object").columns:
        df[c] = df[c].fillna(df[c].mode()[0])

    dup = int(df.duplicated().sum())
    print(f">> Duplikasi     : {dup} baris")
    df = df.drop_duplicates().reset_index(drop=True)

    kol_num = [c for c in df.select_dtypes(include=[np.number]).columns
               if df[c].nunique() > 15]
    Q1, Q3 = df[kol_num].quantile(0.25), df[kol_num].quantile(0.75)
    IQR = Q3 - Q1
    batas = {c: (Q1[c] - 1.5 * IQR[c], Q3[c] + 1.5 * IQR[c]) for c in kol_num}
    for c in kol_num:
        df[c] = np.clip(df[c], *batas[c])
    print(f">> Outlier       : di-capping IQR pada {len(kol_num)} kolom numerik")

    encoder = {}
    for c in df.select_dtypes(include="object").columns:
        le = LabelEncoder()
        df[c] = le.fit_transform(df[c])
        encoder[c] = list(le.classes_)
    label = encoder[TARGET]
    print(f">> Encoding      : {[c for c in encoder if c != TARGET]}")
    print(f">> Target        : 0 = {label[0]} ({(df[TARGET] == 0).sum()}), "
          f"1 = {label[1]} ({(df[TARGET] == 1).sum()})")

    X = df.drop(columns=[TARGET])
    y = df[TARGET]
```

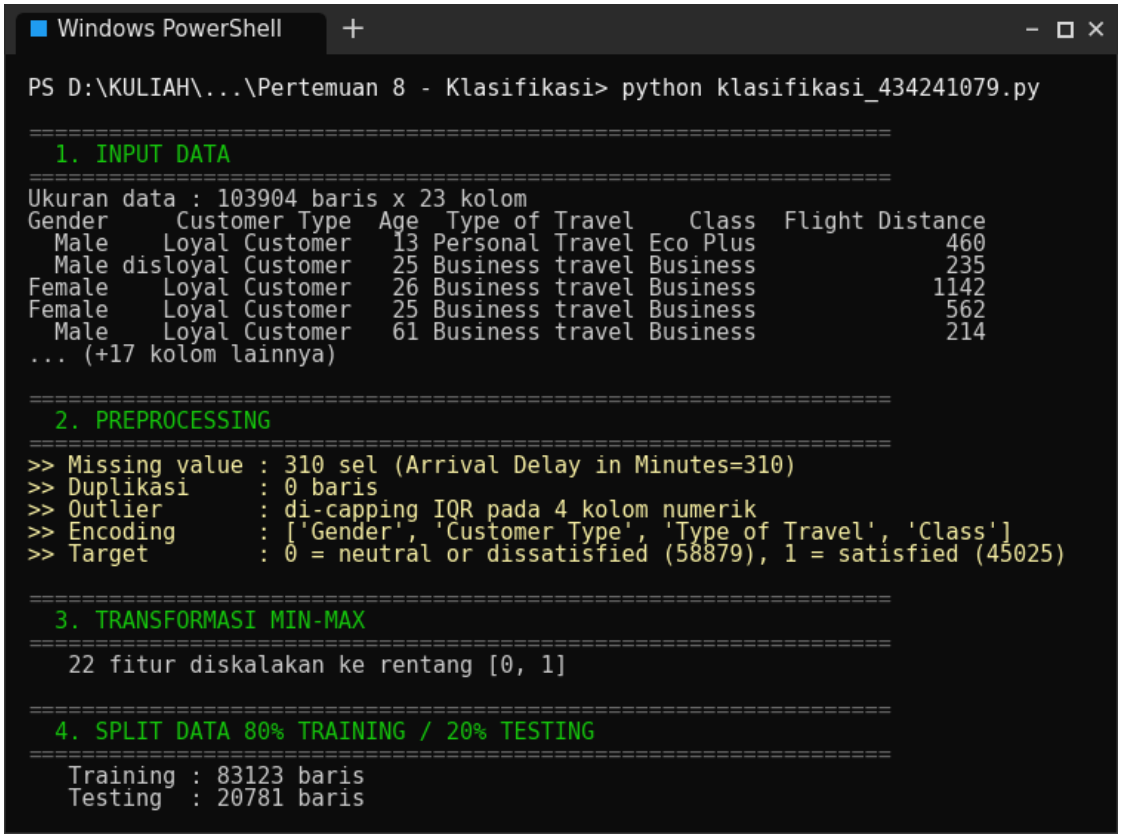
Kode 2. Input data dan preprocessing

py klasifikasi_434241079.py

```
    # ---------------- 3. TRANSFORMASI ----------------
    garis("3. TRANSFORMASI MIN-MAX")
    scaler = MinMaxScaler()
    X_scaled = pd.DataFrame(scaler.fit_transform(X), columns=X.columns)
    print(f"   {X.shape[1]} fitur diskalakan ke rentang "
          f"[{X_scaled.values.min():.0f}, {X_scaled.values.max():.0f}]")

    # ---------------- 4. SPLIT ----------------
    garis("4. SPLIT DATA 80% TRAINING / 20% TESTING")
    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled, y, test_size=0.20, random_state=42, stratify=y)
    print(f"   Training : {len(X_train)} baris")
    print(f"   Testing  : {len(X_test)} baris")
```

Kode 3. Transformasi Min-Max dan split 80/20



Output tahap input, preprocessing, transformasi, dan split

4. Klasifikasi Naive Bayes, SVM, dan Decision Tree

Inti tugas ada di perulangan berikut. Untuk tiap metode seleksi fitur (Chi-Square dan ANOVA F), dipilih 10 fitur teratas, lalu tiga model dilatih pada data training yang sama: **GaussianNB**, **SVC (kernel RBF)**, dan **DecisionTreeClassifier**. Waktu pelatihan dicatat, kemudian setiap model dievaluasi pada data testing melalui fungsi *evaluasi()* yang menghitung confusion matrix, akurasi, presisi, recall, dan F1-Score, sekaligus menyimpan gambar confusion matrix.

py klasifikasi_434241079.py

```
    # ---------------- 5-7. SELEKSI FITUR + KLASIFIKASI + EVALUASI ----------------
    semua_hasil, model_terlatih = [], {}
    for nama_seleksi, fungsi in [("Chi-Square", chi2), ("ANOVA F", f_classif)]:
        garis(f"5. SELEKSI FITUR: {nama_seleksi} ({K_FITUR} fitur teratas)")
        selector = SelectKBest(score_func=fungsi, k=K_FITUR).fit(X_train, y_train)
        fitur = X_train.columns[selector.get_support()].tolist()
        for f in fitur:
            print(f"   - {f}")
        Xtr, Xte = selector.transform(X_train), selector.transform(X_test)

        garis(f"6. KLASIFIKASI & EVALUASI (fitur {nama_seleksi})")
        daftar_model = {
            "Naive Bayes": GaussianNB(),
            "SVM": SVC(kernel="rbf", random_state=42, cache_size=1000),
            "Decision Tree": DecisionTreeClassifier(random_state=42),
        }
        for nama_model, model in daftar_model.items():
            if nama_model == "SVM":
                print(f">> {nama_model} sedang dilatih (bisa 2-3 menit)...")
            mulai = time.time()
            model.fit(Xtr, y_train)
            durasi = time.time() - mulai
            nama = f"{nama_model} + {nama_seleksi}"
            print(f">> {nama}  (waktu latih {durasi:.1f} detik)")
            hasil = evaluasi(nama, y_test, model.predict(Xte), label)
            hasil["Waktu (detik)"] = round(durasi, 1)
            semua_hasil.append(hasil)
            model_terlatih[nama] = (model, selector, fitur)
```

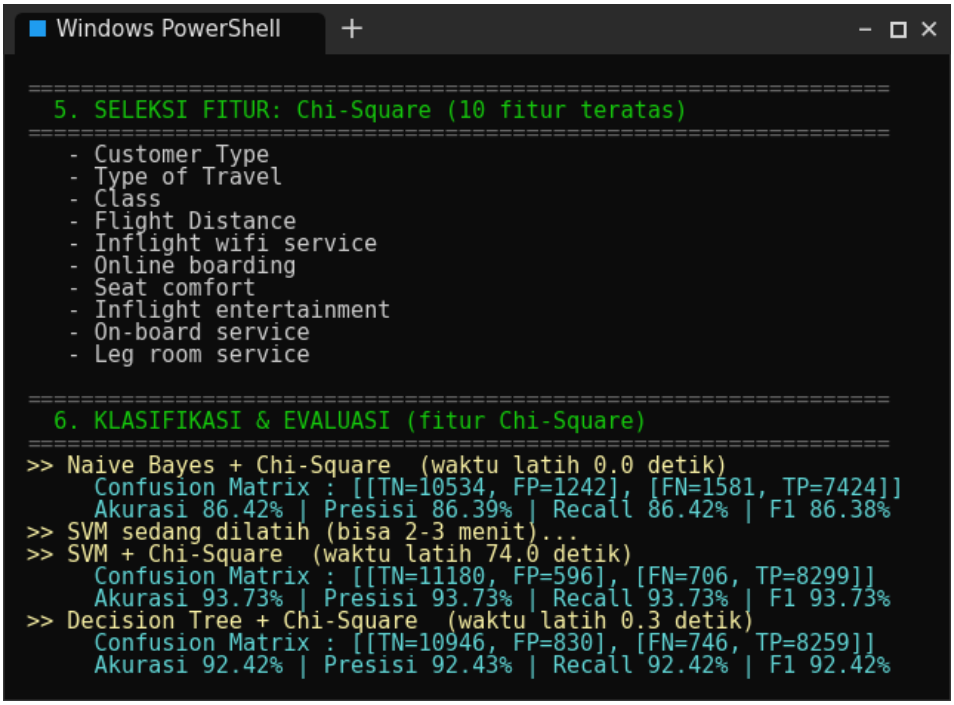
Kode 4. Seleksi fitur dan pelatihan tiga model

py klasifikasi_434241079.py

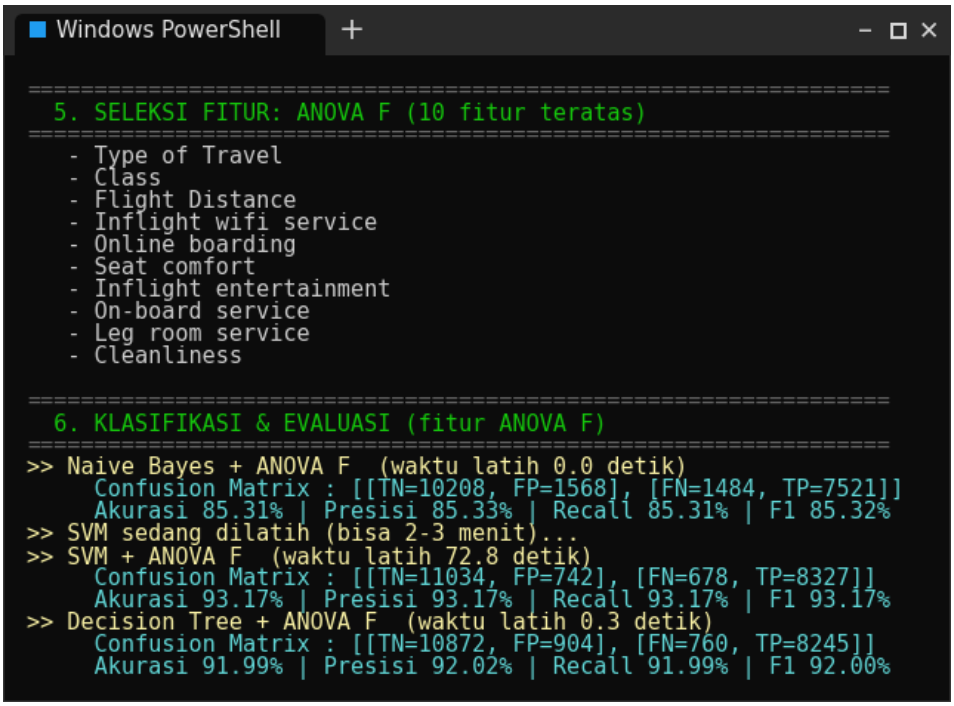
```
def evaluasi(nama, y_test, y_pred, label):
    """Hitung confusion matrix + metrik, cetak, dan simpan gambar CM."""
    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()
    hasil = {
        "Model": nama,
        "Akurasi": accuracy_score(y_test, y_pred),
        "Presisi": precision_score(y_test, y_pred, average="weighted"),
        "Recall": recall_score(y_test, y_pred, average="weighted"),
        "F1": f1_score(y_test, y_pred, average="weighted"),
        "TN": tn, "FP": fp, "FN": fn, "TP": tp,
    }
    print(f"     Confusion Matrix : [[TN={tn}, FP={fp}], [FN={fn}, TP={tp}]]")
    print(f"     Akurasi {hasil['Akurasi']*100:.2f}% | Presisi {hasil['Presisi']*100:.2f}% | "
          f"Recall {hasil['Recall']*100:.2f}% | F1 {hasil['F1']*100:.2f}%")

    plt.figure(figsize=(4.6, 4))
    sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
                xticklabels=label, yticklabels=label)
    plt.xlabel("Prediksi")
    plt.ylabel("Aktual")
    plt.title(f"Confusion Matrix\n{nama}", fontsize=10)
    plt.tight_layout()
    nama_file = nama.lower().replace(" + ", "_").replace(" ", "_").replace("-", "")
    plt.savefig(os.path.join(OUT_DIR, f"cm_{nama_file}.png"), dpi=120)
    plt.close()
    return hasil
```

Kode 5. Fungsi evaluasi (confusion matrix dan metrik)

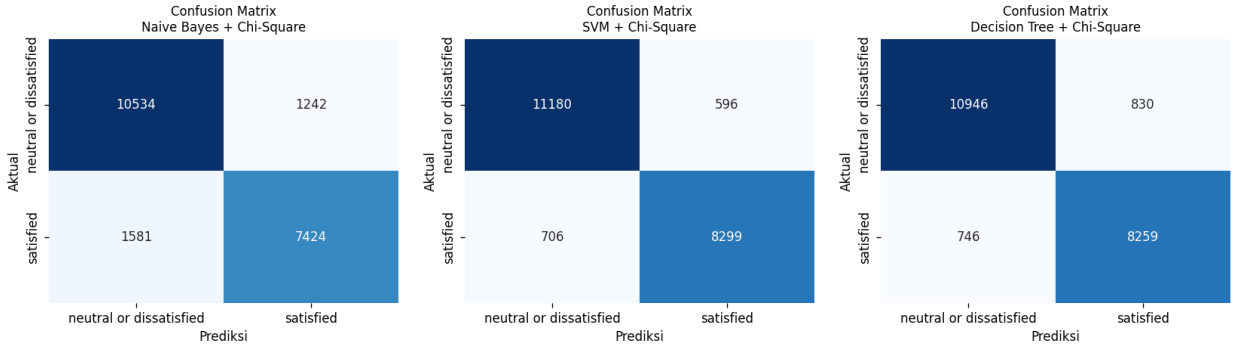


Output klasifikasi dengan fitur hasil Chi-Square



Output klasifikasi dengan fitur hasil ANOVA F

5. Analisis Confusion Matrix



Confusion matrix (seleksi Chi-Square): Naive Bayes, SVM, Decision Tree

Model	TN	FP	FN	TP	Total Salah
Naive Bayes + Chi-Square	10,534	1,242	1,581	7,424	2,823
SVM + Chi-Square	11,180	596	706	8,299	1,302
Decision Tree + Chi-Square	10,946	830	746	8,259	1,576

Tabel 1. Rincian confusion matrix (20.781 data testing)

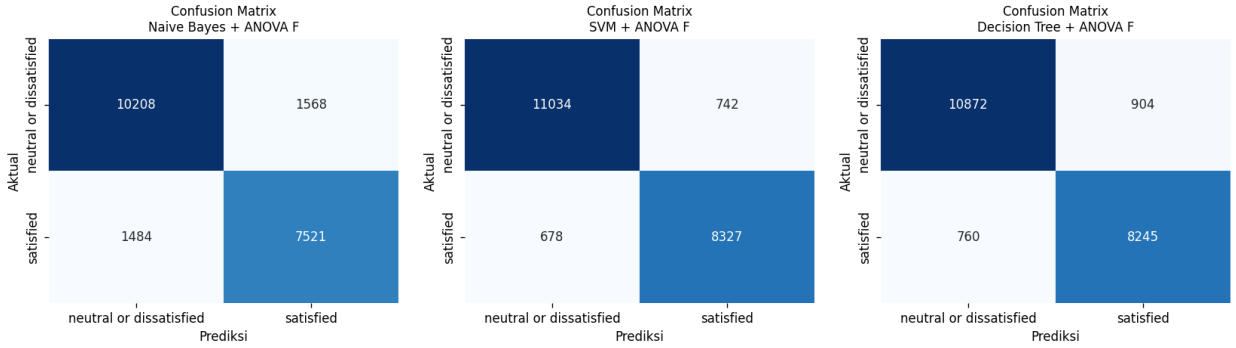
Dengan kelas positif = *satisfied*, confusion matrix dibaca sebagai: **TN** = penumpang tidak puas yang tertebak benar, **FP** = penumpang tidak puas yang salah ditebak puas, **FN** = penumpang puas yang salah ditebak tidak puas, dan **TP** = penumpang puas yang tertebak benar.

- **Naive Bayes** membuat kesalahan terbanyak, yaitu **2,823** data (1,242 FP dan 1,581 FN). Asumsi independen tidak cocok karena rating-rating layanan saling berkaitan (penumpang yang puas dengan kursi cenderung puas juga dengan hiburan), sehingga peluang yang dihitung menjadi bias.

- **SVM** paling sedikit salah: hanya **1,302** data (596 FP dan 706 FN). Kernel RBF mampu menangkap pola non-linear antar-fitur yang tidak bisa ditangkap Naive Bayes.

- **Decision Tree** berada di tengah dengan **1,576** kesalahan (830 FP dan 746 FN), sama persis dengan hasil UTS.

Pada ketiga model, kesalahan tersebar cukup seimbang antara FP dan FN, artinya tidak ada model yang condong berlebihan menebak satu kelas saja.

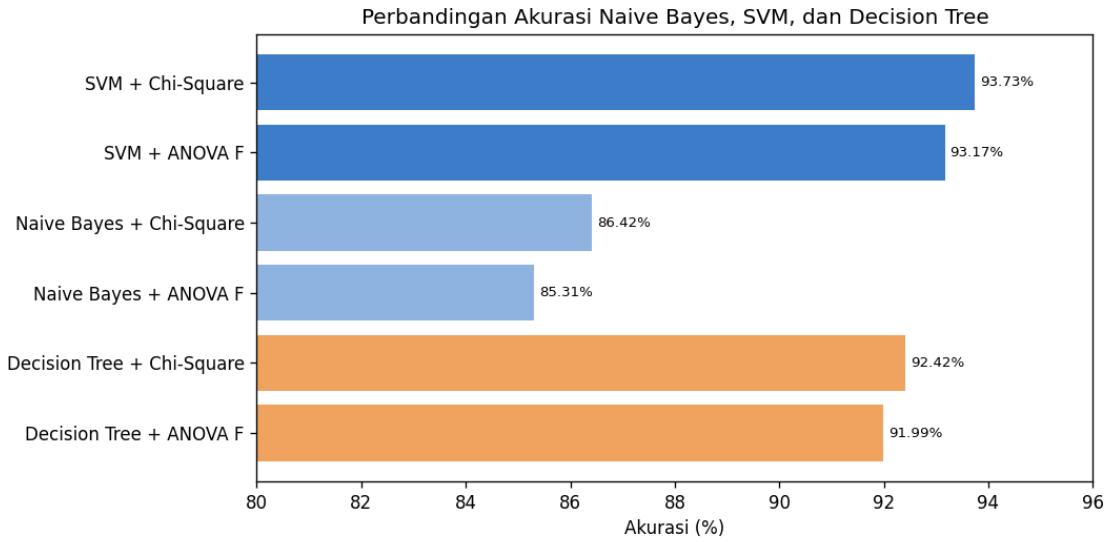


Confusion matrix (seleksi ANOVA F): Naive Bayes, SVM, Decision Tree

6. Perbandingan Kinerja

Model	Akurasi	Presisi	Recall	F1-Score	Waktu Latih
SVM + Chi-Square	93.73%	93.73%	93.73%	93.73%	74.0 s
SVM + ANOVA F	93.17%	93.17%	93.17%	93.17%	72.8 s
Decision Tree + Chi-Square	92.42%	92.43%	92.42%	92.42%	0.3 s
Decision Tree + ANOVA F	91.99%	92.02%	91.99%	92.00%	0.3 s
Naive Bayes + Chi-Square	86.42%	86.39%	86.42%	86.38%	0.0 s
Naive Bayes + ANOVA F	85.31%	85.33%	85.31%	85.32%	0.0 s

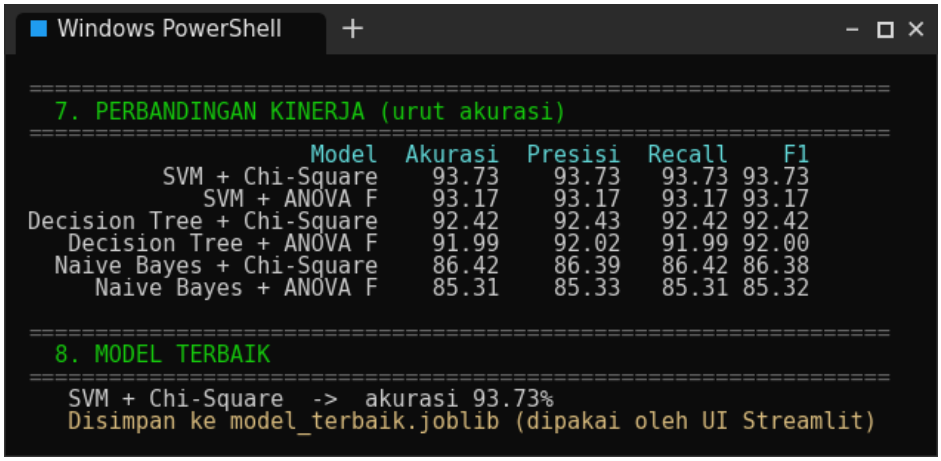
Tabel 2. Perbandingan seluruh kombinasi model dan seleksi fitur (urut akurasi)



Perbandingan akurasi ketiga metode

Urutan kinerja konsisten di kedua metode seleksi fitur: **SVM > Decision Tree > Naive Bayes**. Akurasi tertinggi diraih **SVM + Chi-Square** sebesar **93,73%**, unggul dari Decision Tree (92,42%) dan jauh di atas Naive Bayes (86,42%). Seleksi Chi-Square juga sedikit lebih baik dari ANOVA F untuk ketiga model.

Namun keunggulan SVM dibayar dengan waktu: SVM butuh sekitar **74 detik** untuk dilatih, sementara Decision Tree kurang dari 1 detik dan Naive Bayes hampir instan. Untuk data sebesar ini, Decision Tree tetap menjadi pilihan yang sangat efisien, tetapi bila yang diutamakan adalah ketepatan prediksi, SVM adalah model terbaik. Karena itu SVM + Chi-Square dipilih untuk UI.



Output perbandingan dan penyimpanan model terbaik

7. UI/UX Streamlit (Model Terbaik: SVM)

Model terbaik disimpan ke file **model_terbaik.joblib** bersama scaler, selector, encoder, dan batas outlier, sehingga UI tidak perlu melatih ulang SVM. Aplikasi **app_streamlit_434241079.py** membaca file tersebut lalu menampilkan form berisi 10 fitur hasil seleksi Chi-Square. Input dari pengguna diproses dengan urutan yang sama seperti saat pelatihan (encoding → capping → Min-Max → seleksi fitur) sebelum diprediksi oleh SVM. Hasil prediksi diperbarui otomatis setiap kali input diubah.

py klasifikasi_434241079.py

```
    # ---------------- 8. PERBANDINGAN ----------------
    garis("7. PERBANDINGAN KINERJA (urut akurasi)")
    tabel = pd.DataFrame(semua_hasil).sort_values("Akurasi", ascending=False)
    tampil = tabel[["Model", "Akurasi", "Presisi", "Recall", "F1"]].copy()
    for c in ["Akurasi", "Presisi", "Recall", "F1"]:
        tampil[c] = (tampil[c] * 100).round(2)
    print(tampil.to_string(index=False))
    tabel.to_csv(os.path.join(OUT_DIR, "perbandingan_model.csv"), index=False)

    plt.figure(figsize=(9, 4.5))
    warna = {"Naive Bayes": "#8fb3e0", "SVM": "#3d7cc9", "Decision Tree": "#f0a35e"}
    urut = tabel.sort_values("Model")
    bar = plt.barh(urut["Model"], urut["Akurasi"] * 100,
                   color=[warna[m.split(" + ")[0]] for m in urut["Model"]])
    plt.bar_label(bar, fmt="%.2f%%", padding=3, fontsize=8)
    plt.xlim(80, 96)
    plt.xlabel("Akurasi (%)")
    plt.title("Perbandingan Akurasi Naive Bayes, SVM, dan Decision Tree")
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "perbandingan_akurasi.png"), dpi=120)
    plt.close()

    # ---------------- 9. SIMPAN MODEL TERBAIK ----------------
    terbaik = tabel.iloc[0]["Model"]
    model, selector, fitur = model_terlatih[terbaik]
    bundle = {
        "nama_model": terbaik,
        "akurasi": float(tabel.iloc[0]["Akurasi"]),
        "model": model,
        "scaler": scaler,
        "selector": selector,
        "urutan_fitur": list(X.columns),
        "fitur_terpilih": fitur,
        "encoder": {k: v for k, v in encoder.items() if k != TARGET},
        "label": label,
        "default": X.median().to_dict(),
        "batas_outlier": batas,
        "rentang": {c: (float(X[c].min()), float(X[c].max())) for c in X.columns},
        "perbandingan": tampil.to_dict("records"),
    }
    joblib.dump(bundle, os.path.join(BASE_DIR, "model_terbaik.joblib"))
    garis("8. MODEL TERBAIK")
    print(f"   {terbaik}  ->  akurasi {tabel.iloc[0]['Akurasi'] * 100:.2f}%")
    print("   Disimpan ke model_terbaik.joblib (dipakai oleh UI Streamlit)")
```

Kode 6. Perbandingan dan penyimpanan model terbaik

py app_streamlit_434241079.py

```python
@st.cache_resource
def muat_model():
    return joblib.load(PATH_MODEL)


def isi_contoh(bundle, kunci):
    """Isi semua input dengan nilai contoh (disimpan di session_state)."""
    c = CONTOH[kunci]
    for f in bundle["fitur_terpilih"]:
        if f in bundle["encoder"]:
            st.session_state[f] = c["kategori"].get(f, bundle["encoder"][f][0])
        elif bundle["rentang"][f][1] <= 5:
            st.session_state[f] = c["rating"]
        else:
            lo, hi = bundle["rentang"][f]
            st.session_state[f] = float(min(max(c["angka"].get(f, (lo + hi) / 2), lo), hi))


def prediksi(bundle, masukan):
    """Ubah input menjadi baris data, lalu: capping -> Min-Max -> seleksi fitur -> model."""
    baris = dict(bundle["default"])
    for f, nilai in masukan.items():
        if f in bundle["encoder"]:
            baris[f] = bundle["encoder"][f].index(nilai)
        else:
            lo, hi = bundle["batas_outlier"].get(f, (float("-inf"), float("inf")))
            baris[f] = min(max(float(nilai), lo), hi)
    data = pd.DataFrame([baris])[bundle["urutan_fitur"]]
    data = pd.DataFrame(bundle["scaler"].transform(data), columns=bundle["urutan_fitur"])
    data = bundle["selector"].transform(data)
    kelas = int(bundle["model"].predict(data)[0])
    skor = float(bundle["model"].decision_function(data)[0]) \
        if hasattr(bundle["model"], "decision_function") else None
    return bundle["label"][kelas], skor
```

Kode 7. app_streamlit: memuat model dan fungsi prediksi

py app_streamlit_434241079.py

```python
# ====================== TAMPILAN ======================
st.set_page_config(page_title="Prediksi Kepuasan Penumpang", page_icon=":airplane:", layout="wide")

if not os.path.exists(PATH_MODEL):
    st.error("File model_terbaik.joblib belum ada. Jalankan dulu: python klasifikasi_434241079.py")
    st.stop()

bundle = muat_model()
fitur = bundle["fitur_terpilih"]

contoh_url = st.query_params.get("contoh")
if contoh_url in CONTOH and st.session_state.get("_contoh") != contoh_url:
    isi_contoh(bundle, contoh_url)
    st.session_state["_contoh"] = contoh_url

st.title(":airplane: Prediksi Kepuasan Penumpang Maskapai")
st.caption(f"Model: **{bundle['nama_model']}** · akurasi data testing "
           f"**{bundle['akurasi'] * 100:.2f}%** · Kelompok 5 TI-C5")

with st.sidebar:
    st.header("Tentang Model")
    st.write(f"Model terpilih: **{bundle['nama_model']}**")
    st.metric("Akurasi (data testing)", f"{bundle['akurasi'] * 100:.2f}%")
    st.write("Perbandingan dengan metode lain:")
    st.dataframe(pd.DataFrame(bundle["perbandingan"])[["Model", "Akurasi"]],
                 hide_index=True, width="stretch")
    st.divider()
    st.write("Isi cepat dengan contoh:")
    if st.button("Penumpang puas", icon=":material/sentiment_satisfied:", width="stretch"):
        isi_contoh(bundle, "puas")
    if st.button("Penumpang tidak puas", icon=":material/sentiment_dissatisfied:",
                 width="stretch"):
        isi_contoh(bundle, "tidak")
```

Kode 8. app_streamlit: judul dan sidebar

py app_streamlit_434241079.py

```python
kol_kiri, kol_kanan = st.columns(2, gap="large")
masukan = {}

with kol_kiri:
    st.subheader(":briefcase: Profil Perjalanan")
    for f in fitur:
        if f in bundle["encoder"]:
            pilihan = bundle["encoder"][f]
            if f not in st.session_state:
                st.session_state[f] = pilihan[0]
            masukan[f] = st.selectbox(LABEL_FITUR.get(f, f), pilihan, key=f,
                                      format_func=lambda v: LABEL_NILAI.get(v, v))
        elif bundle["rentang"][f][1] > 5:
            lo, hi = bundle["rentang"][f]
            if f not in st.session_state:
                st.session_state[f] = float(bundle["default"][f])
            masukan[f] = st.number_input(LABEL_FITUR.get(f, f), min_value=float(lo),
                                         max_value=float(hi), step=50.0, key=f)

with kol_kanan:
    st.subheader(":star: Penilaian Layanan")
    st.caption("Skala 0–5 · 0 = tidak dinilai, 1 = sangat buruk, 5 = sangat baik")
    for f in fitur:
        if f not in bundle["encoder"] and bundle["rentang"][f][1] <= 5:
            if f not in st.session_state:
                st.session_state[f] = 3
            masukan[f] = st.slider(LABEL_FITUR.get(f, f), 0, 5, key=f)

st.divider()
hasil, skor = prediksi(bundle, masukan)
puas = hasil == "satisfied"
with st.container(border=True):
    st.subheader("Hasil Prediksi")
    if puas:
        st.success("### PUAS (satisfied)\nPenumpang ini diprediksi **puas** dengan penerbangannya.",
                   icon=":material/sentiment_satisfied:")
    else:
        st.error("### NETRAL / TIDAK PUAS\nPenumpang ini diprediksi **netral atau tidak puas**.",
                 icon=":material/sentiment_dissatisfied:")
    if skor is not None:
        st.caption(f"Skor keputusan SVM: {skor:+.2f} — positif cenderung *puas*, negatif cenderung "
                   "*tidak puas*; makin jauh dari 0 makin yakin.")
```

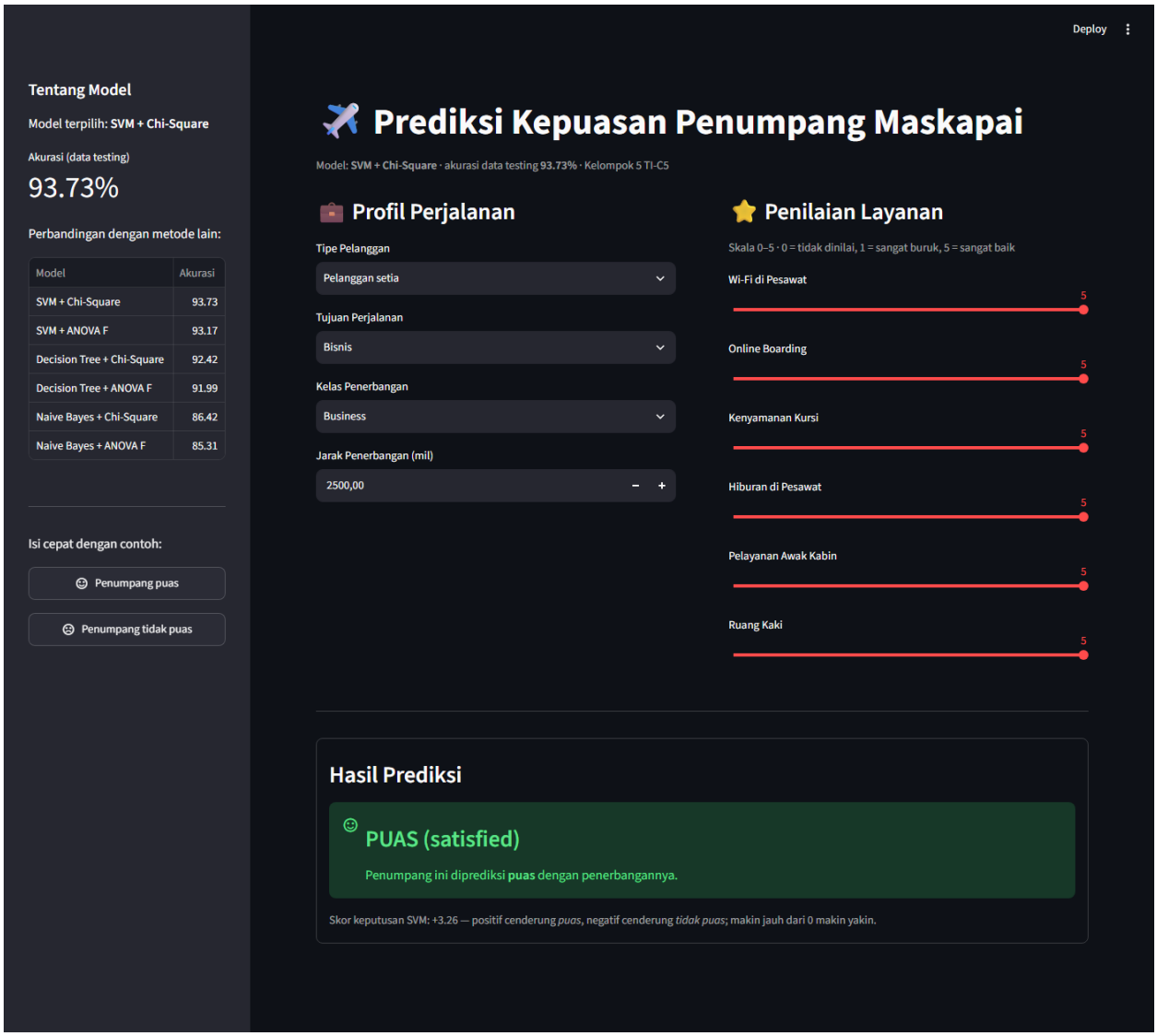
Kode 9. app_streamlit: form input dan hasil prediksi

Cara menjalankan aplikasi:

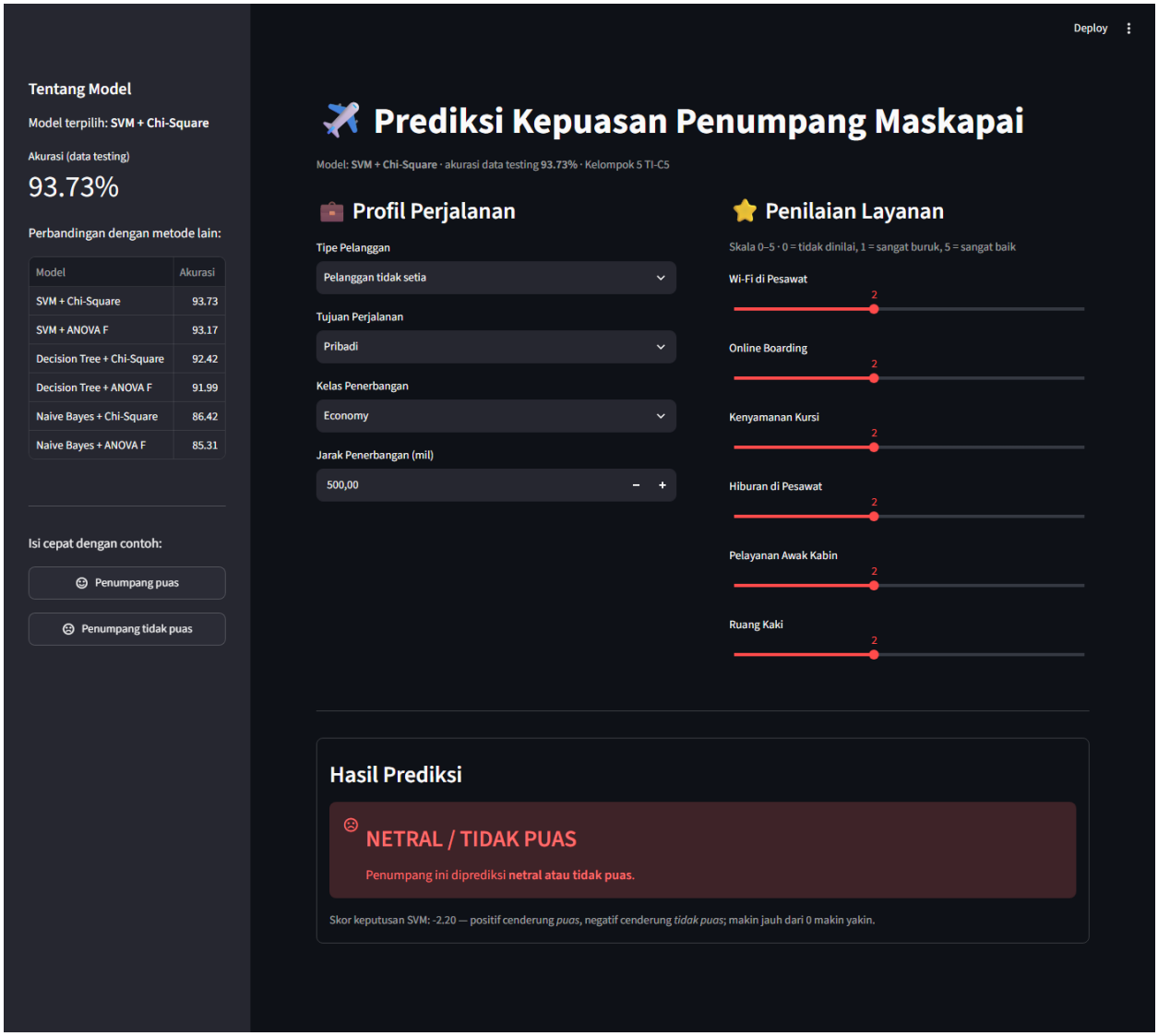
```
pip install streamlit scikit-learn pandas joblib
python klasifikasi_434241079.py          # melatih & menyimpan model
streamlit run app_streamlit_434241079.py # membuka UI di browser
```

Tampilan Aplikasi

Tampilan UI terdiri dari: **(1)** judul dan info model beserta akurasinya, **(2)** sidebar berisi perbandingan akurasi ketiga metode dan tombol *isi cepat* contoh penumpang, **(3)** kolom *Profil Perjalanan* (tipe pelanggan, tujuan perjalanan, kelas, jarak), **(4)** kolom *Penilaian Layanan* berupa slider 0–5, dan **(5)** kartu *Hasil Prediksi* berwarna hijau untuk *puas* atau merah untuk *netral/tidak puas*, lengkap dengan skor keputusan SVM.



UI: contoh penumpang yang diprediksi PUAS



UI: contoh penumpang yang diprediksi NETRAL / TIDAK PUAS

8. Kesimpulan

Ketiga metode berhasil diterapkan dengan tahapan yang sama seperti UTS. **SVM (kernel RBF)** memberikan kinerja terbaik dengan akurasi **93,73%** (seleksi Chi-Square), disusul **Decision Tree** (92,42%) dan **Naive Bayes** (86,42%). Naive Bayes tertinggal karena asumsi independensi fiturnya tidak terpenuhi, sedangkan SVM unggul berkat kemampuannya memodelkan pola non-linear meski waktu pelatihannya paling lama. Model SVM terbaik kemudian diwujudkan dalam aplikasi Streamlit yang dapat memprediksi kepuasan penumpang secara interaktif.

9. Lanjutan: Klasifikasi Tanpa Transformasi dan Seleksi Fitur (Ori)

Pada pertemuan lanjutan, dosen meminta tiga metode klasifikasi yang sama dijalankan ulang **tanpa melewati tahap transformasi dan seleksi fitur**. Artinya, setelah data di-split 80% training dan 20% testing, data langsung dipakai untuk melatih Naive Bayes, SVM, dan Decision Tree. Versi ini kami sebut **Ori** (original), lalu dibandingkan dengan versi sebelumnya yang memakai Min-Max dan seleksi fitur, untuk melihat apakah kedua tahap tersebut benar-benar membantu.

Tahap preprocessing tetap dijalankan, karena missing value harus diisi dan kolom kategorikal harus di-encode agar data bisa diproses model. Pembagian data juga memakai *random_state* dan *stratify* yang sama, sehingga baris data training dan testing persis sama dengan skenario sebelumnya. Dengan begitu perbandingannya adil: yang berbeda hanya ada atau tidaknya Min-Max dan seleksi fitur.

py klasifikasi_434241079.py

```
212     # ---------------- 9. LANJUTAN: TANPA TRANSFORMASI & SELEKSI FITUR (ORI) ----------------
213     garis("9. LANJUTAN: TANPA TRANSFORMASI & SELEKSI FITUR (ORI)")
214     # data hasil preprocessing langsung di-split (tanpa Min-Max, tanpa seleksi fitur);
215     # random_state & stratify sama -> baris training/testing identik dengan skenario di atas
216     Xo_train, Xo_test, yo_train, yo_test = train_test_split(
217         X, y, test_size=0.20, random_state=42, stratify=y)
218     print(f"   Semua {X.shape[1]} fitur dipakai, nilai asli (tidak dinormalisasi)")
219     print(f"   Training : {len(Xo_train)} baris | Testing : {len(Xo_test)} baris")
220     daftar_ori = {
221         "Naive Bayes": GaussianNB(),
222         "SVM": SVC(kernel="rbf", random_state=42, cache_size=1000),
223         "Decision Tree": DecisionTreeClassifier(random_state=42),
224     }
225     for nama_model, model in daftar_ori.items():
226         if nama_model == "SVM":
227             print(f">> {nama_model} sedang dilatih tanpa normalisasi (bisa lebih lama)...")
228         mulai = time.time()
229         model.fit(Xo_train, yo_train)
230         durasi = time.time() - mulai
231         nama = f"{nama_model} (Ori)"
232         print(f">> {nama}  (waktu latih {durasi:.1f} detik)")
233         hasil = evaluasi(nama, yo_test, model.predict(Xo_test), label)
234         hasil["Waktu (detik)"] = round(durasi, 1)
235         semua_hasil.append(hasil)
```

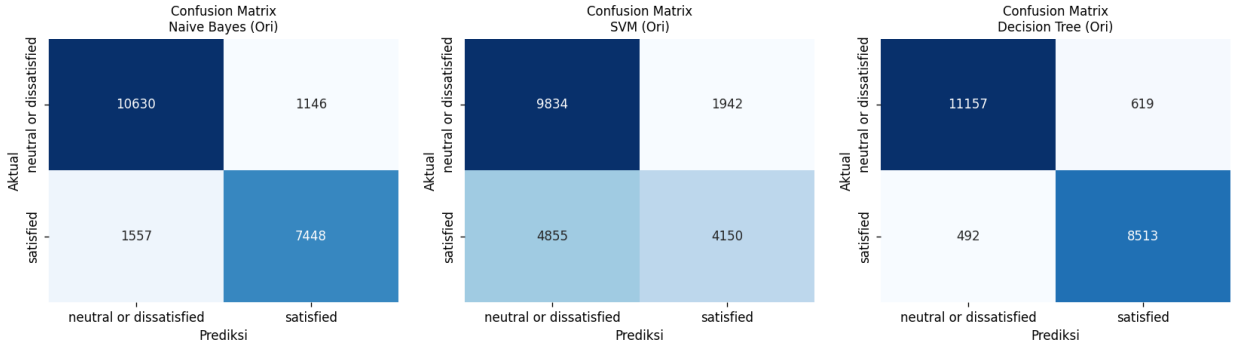
Kode 10. Skenario Ori: split lalu langsung klasifikasi

Windows PowerShell

```
============================================================
  9. LANJUTAN: TANPA TRANSFORMASI & SELEKSI FITUR (ORI)
============================================================
   Semua 22 fitur dipakai, nilai asli (tidak dinormalisasi)
   Training : 83123 baris | Testing : 20781 baris
>> Naive Bayes (Ori)  (waktu latih 0.0 detik)
     Confusion Matrix : [[TN=10630, FP=1146], [FN=1557, TP=7448]]
     Akurasi 86.99% | Presisi 86.98% | Recall 86.99% | F1 86.95%
>> SVM sedang dilatih tanpa normalisasi (bisa lebih lama)...
>> SVM (Ori)  (waktu latih 561.1 detik)
     Confusion Matrix : [[TN=9834, FP=1942], [FN=4855, TP=4150]]
     Akurasi 67.29% | Presisi 67.46% | Recall 67.29% | F1 65.94%
>> Decision Tree (Ori)  (waktu latih 0.7 detik)
     Confusion Matrix : [[TN=11157, FP=619], [FN=492, TP=8513]]
     Akurasi 94.65% | Presisi 94.67% | Recall 94.65% | F1 94.66%
```

Output klasifikasi skenario Ori

Kode lanjutan ditambahkan pada bagian akhir file **klasifikasi_434241079.py** yang sama. Variabel **X** di sini adalah data hasil preprocessing yang belum dinormalisasi, berisi **22 fitur** lengkap, langsung dibagi dengan *train_test_split* lalu dipakai melatih ketiga model. Fungsi *evaluasi()* yang sama dipakai kembali agar cara penilaiannya identik.



Confusion matrix skenario Ori: Naive Bayes, SVM, Decision Tree

Model	TN	FP	FN	TP	Total Salah
Naive Bayes (Ori)	10,630	1,146	1,557	7,448	2,703
SVM (Ori)	9,834	1,942	4,855	4,150	6,797
Decision Tree (Ori)	11,157	619	492	8,513	1,111

Tabel 3. Rincian confusion matrix skenario Ori (20.781 data testing)

Hasil ketiga model pada skenario Ori:

- **Naive Bayes (Ori)** mencapai **86,99%** (naik 0,58 poin dibanding versi Chi-Square). GaussianNB menghitung rata-rata dan simpangan baku per fitur, sehingga tidak terpengaruh normalisasi; perbedaannya murni karena sekarang memakai 22 fitur, bukan 10.

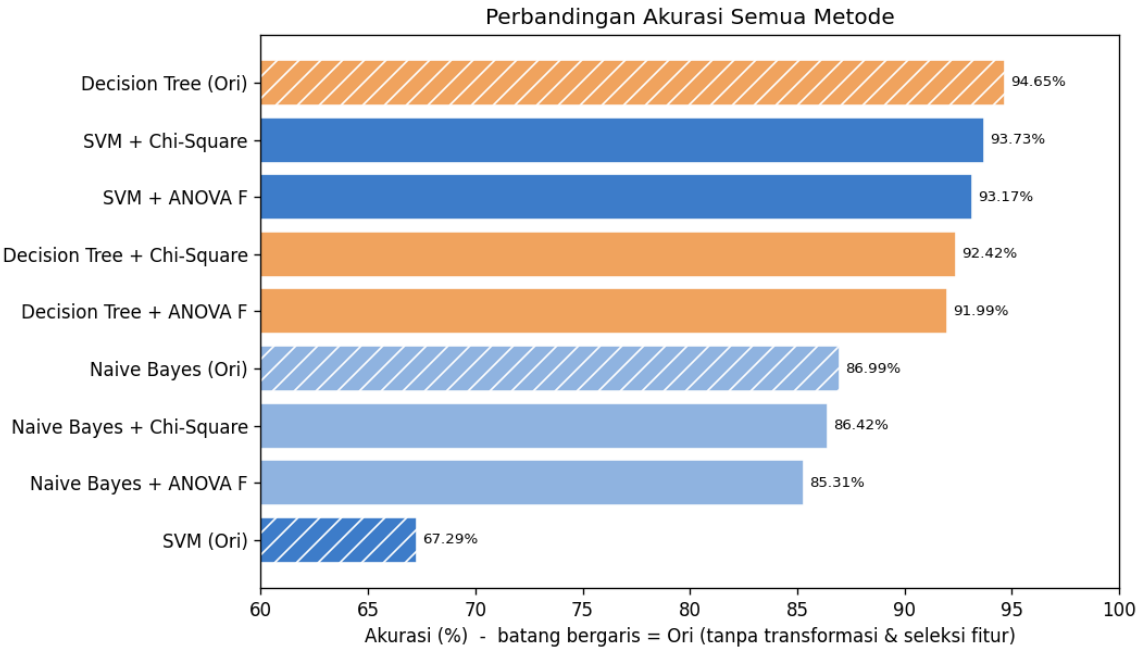
- **SVM (Ori)** justru **turun** menjadi **67,29%** dari 93,73%, dengan **6,797** kesalahan, dan waktu latihnya melonjak menjadi sekitar **561 detik**. SVM kernel RBF bekerja dengan mengukur *jarak* antar-data. Tanpa Min-Max, kolom bernilai besar seperti *Flight Distance* (31 sampai 3.736) mendominasi jarak tersebut, sementara rating layanan (0 sampai 5) nyaris tidak berpengaruh. Hasilnya, SVM kehilangan banyak informasi dan juga makin lambat mencapai konvergensi. Confusion matrix-nya memperlihatkan **4,855** penumpang puas tertebak tidak puas (FN), jauh lebih banyak dari FP (1,942), artinya model condong menebak kelas mayoritas *neutral or dissatisfied*.

- **Decision Tree (Ori)** menjadi yang terbaik di skenario ini dengan **94,65%** (naik 2,24 poin dibanding versi Chi-Square) dan hanya **1,111** kesalahan. Decision Tree memecah data berdasarkan ambang nilai, jadi skala fitur tidak berpengaruh sama sekali. Kenaikannya datang dari 12 fitur tambahan (misalnya *Age*, *Departure Delay*, *Food and drink*, *Cleanliness*) yang ternyata masih membawa informasi berguna tetapi terbuang saat seleksi fitur.

10. Perbandingan Semua Metode

Model	Skenario	Akurasi	Presisi	Recall	F1-Score	Waktu Latih
Decision Tree (Ori)	Ori	94.65%	94.67%	94.65%	94.66%	0.7 s
SVM + Chi-Square	Min-Max + seleksi fitur	93.73%	93.73%	93.73%	93.73%	74.0 s
SVM + ANOVA F	Min-Max + seleksi fitur	93.17%	93.17%	93.17%	93.17%	72.8 s
Decision Tree + Chi-Square	Min-Max + seleksi fitur	92.42%	92.43%	92.42%	92.42%	0.3 s
Decision Tree + ANOVA F	Min-Max + seleksi fitur	91.99%	92.02%	91.99%	92.00%	0.3 s
Naive Bayes (Ori)	Ori	86.99%	86.98%	86.99%	86.95%	0.0 s
Naive Bayes + Chi-Square	Min-Max + seleksi fitur	86.42%	86.39%	86.42%	86.38%	0.0 s
Naive Bayes + ANOVA F	Min-Max + seleksi fitur	85.31%	85.33%	85.31%	85.32%	0.0 s
SVM (Ori)	Ori	67.29%	67.46%	67.29%	65.94%	561.1 s

Tabel 4. Perbandingan semua metode yang pernah dicoba (urut akurasi)



Perbandingan akurasi semua metode (batang bergaris = Ori)

Metode	Ori	Chi-Square	ANOVA F	Selisih Ori vs terbaik (poin)
Naive Bayes	86.99%	86.42%	85.31%	+0.58
SVM	67.29%	93.73%	93.17%	-26.44
Decision Tree	94.65%	92.42%	91.99%	+2.24

Tabel 5. Akurasi tiap metode per skenario

Tabel di atas menggabungkan **seluruh metode yang pernah dicoba** pada dataset ini, termasuk Decision Tree + Chi-Square yang hasilnya sama dengan UTS. Beberapa hal yang terlihat:

- Akurasi tertinggi secara keseluruhan diraih **Decision Tree (Ori)** sebesar **94,65%**.

- **Transformasi Min-Max** sangat menentukan untuk **SVM**, karena SVM berbasis jarak. Dengan Min-Max dan seleksi fitur, SVM mencapai 93,73% dalam sekitar 74 detik; tanpa keduanya hanya 67,29% dan butuh sekitar 561 detik. Sebaliknya, Naive Bayes dan Decision Tree tidak sensitif terhadap skala, sehingga normalisasi tidak mengubah hasilnya.

- **Seleksi fitur** mengurangi data dari 22 menjadi 10 fitur. Pada dataset ini, pengurangan tersebut membuat Naive Bayes dan Decision Tree sedikit kehilangan informasi, sehingga versi Ori justru lebih akurat: Decision Tree naik dari 92,42% menjadi 94,65%.

- Jadi kedua tahap tersebut **tidak selalu meningkatkan akurasi**. Manfaatnya bergantung pada algoritma yang dipakai dan seberapa banyak informasi yang ikut terbuang.

py klasifikasi_434241079.py

```
    # ---------------- 10. PERBANDINGAN SEMUA METODE ----------------
    garis("10. PERBANDINGAN SEMUA METODE (urut akurasi)")
    tabel_semua = pd.DataFrame(semua_hasil).sort_values("Akurasi", ascending=False)
    tabel_semua.insert(1, "Skenario", ["Ori (tanpa transformasi & seleksi)" if "(Ori)" in m
                                       else "Min-Max + seleksi fitur" for m in tabel_semua["Model"]])
    tampil_semua = tabel_semua[["Model", "Akurasi", "Presisi", "Recall", "F1", "Waktu (detik)"]].copy()
    for c in ["Akurasi", "Presisi", "Recall", "F1"]:
        tampil_semua[c] = (tampil_semua[c] * 100).round(2)
    print(tampil_semua.to_string(index=False))
    tabel_semua.to_csv(os.path.join(OUT_DIR, "perbandingan_semua.csv"), index=False)

    plt.figure(figsize=(9, 5.2))
    urut = tabel_semua.sort_values("Akurasi")
    bar = plt.barh(urut["Model"], urut["Akurasi"] * 100, edgecolor="white",
                   color=[warna[m.split(" + ")[0].split(" (")[0]] for m in urut["Model"]])
    for batang, m in zip(bar, urut["Model"]):
        if "(Ori)" in m:
            batang.set_hatch("//")
    plt.bar_label(bar, fmt="%.2f%%", padding=3, fontsize=8)
    plt.xlim(60, 100)
    plt.xlabel("Akurasi (%)  -  batang bergaris = Ori (tanpa transformasi & seleksi fitur)")
    plt.title("Perbandingan Akurasi Semua Metode")
    plt.tight_layout()
    plt.savefig(os.path.join(OUT_DIR, "perbandingan_semua.png"), dpi=120)
    plt.close()

    juara = tabel_semua.iloc[0]
    print(f"\n   Akurasi tertinggi keseluruhan: {juara['Model']} ({juara['Akurasi'] * 100:.2f}%)")
```

Kode 11. Tabel dan grafik perbandingan semua metode

Windows PowerShell

```
============================================================
  10. PERBANDINGAN SEMUA METODE (urut akurasi)
============================================================
                    Model  Akurasi  Presisi  Recall     F1  Waktu (detik)
        Decision Tree (Ori)    94.65    94.67   94.65 94.66            0.7
           SVM + Chi-Square    93.73    93.73   93.73 93.73           74.0
              SVM + ANOVA F    93.17    93.17   93.17 93.17           72.8
Decision Tree + Chi-Square     92.42    92.43   92.42 92.42            0.3
   Decision Tree + ANOVA F     91.99    92.02   91.99 92.00            0.3
         Naive Bayes (Ori)     86.99    86.98   86.99 86.95            0.0
  Naive Bayes + Chi-Square     86.42    86.39   86.42 86.38            0.0
     Naive Bayes + ANOVA F     85.31    85.33   85.31 85.32            0.0
                 SVM (Ori)     67.29    67.46   67.29 65.94          561.1

   Akurasi tertinggi keseluruhan: Decision Tree (Ori) (94.65%)
```

Output perbandingan semua metode

11. Kesimpulan Lanjutan

Dari seluruh percobaan, **Decision Tree (Ori)** memberikan akurasi tertinggi, yaitu **94,65%**. Perbandingan skenario Ori dengan skenario Min-Max + seleksi fitur menunjukkan bahwa langkah preprocessing lanjutan sebaiknya dipilih sesuai karakter algoritma: normalisasi wajib untuk metode berbasis jarak seperti SVM, sedangkan untuk Decision Tree dan Naive Bayes, mempertahankan semua fitur informatif justru lebih menguntungkan pada data ini.

UI Streamlit pada bagian 7 dibangun dari skenario pertama (SVM + Chi-Square). Berdasarkan perbandingan lengkap ini, model yang paling layak dipakai untuk UI adalah **Decision Tree (Ori)**, yang selain paling akurat juga paling cepat dilatih.